
Trabalho Final - Relatório

Parte IV

Grupo: Spotify Explorer

Disciplina: Banco de Dados I

Introdução

O presente trabalho documenta o ciclo completo de desenvolvimento de um sistema de banco de dados relacional e sua respectiva integração a um ecossistema de software estruturado. O objetivo central do projeto foi conceber, normalizar e implementar uma base de dados robusta fundamentada em informações musicais massivas extraídas do *Spotify Tracks Dataset*, permitindo a organização, a consistência e a consulta otimizada de dados correlacionados a músicas, artistas, álbuns e gêneros musicais.

O desenvolvimento do projeto seguiu rigorosamente as etapas clássicas de modelagem de dados: iniciando pela abstração de um Modelo Entidade-Relacionamento (Conceitual) baseado nas regras de negócio e na estrutura do *dataset* selecionado; evoluindo para o mapeamento lógico e aplicação das formas normais; e culminando na sua implementação física em um Sistema Gerenciador de Banco de Dados (SGBD) relacional.

Para validar e explorar o banco de dados construído, foi desenvolvida a aplicação Web denominada Spotify Explorer. Longe de se limitar a uma interface local simples, o produto final foi consolidado como um sistema distribuído baseado em uma arquitetura de três camadas (*3-tier architecture*). O ecossistema é composto por uma camada de apresentação independente (construída em Node.js com Vite), uma camada de lógica de negócios atuando como API RESTful (desenvolvida em Python com Flask e SQLAlchemy) e a camada de persistência de dados (MySQL).

Visando simular um ambiente de produção real de mercado, a solução foi totalmente implantada na nuvem. O SGBD relacional encontra-se hospedado de forma gerenciada na plataforma Aiven, enquanto as camadas de aplicação (frontend e backend) operam de forma contínua na plataforma Render. Dessa forma, o projeto demonstra na prática a viabilidade de integração entre a teoria de bancos de dados relacionais, o desenvolvimento de APIs modernas e a infraestrutura de computação em nuvem, servindo como uma simulação fiel dos desafios encontrados no cotidiano profissional da área de tecnologia.

Datasets

=====

O conjunto de dados utilizado no projeto foi o Spotify Tracks Dataset, obtido através da plataforma Kaggle. O dataset foi disponibilizado no formato CSV, e contém uma grande quantidade de músicas catalogadas juntamente com demais informações listadas anteriormente.

O dataset foi escolhido por possuir informações suficientes para a construção de um modelo conceitual coerente, bem como para testar o bom funcionamento do banco de dados, posteriormente.

Spotify Tracks Dataset

Os campos analisados no dataset foram:

```
track_id
artists
album_name
track_name
popularity
duration_ms
explicit
danceability
energy
key
loudness
mode
speechiness
acousticness
instrumentalness
liveness
valence
tempo
time_signature
track_genre
```

Após a análise dos dados, foram identificadas quatro entidades principais para a modelagem do banco de dados:

- Música
- Artista
- Álbum
- Gênero

Os tipos presumidos dos dados presentes no dataset incluem:

Campo	Tipo Presumido	Valores Típicos
-------	----------------	-----------------

track_id	String	Identificadores únicos de músicas
artists	String	Nome de artistas ou múltiplos artistas separados por ";"
album_name	String	Nome de álbuns musicais
track_name	String	Nome das músicas
popularity	Integer	Valores entre 0 e 100
duration_ms	Integer	Duração da música em milissegundos
track_genre	String	Categorias de gêneros musicais

Demais campos no dataset representam características técnicas e métricas de áudio das músicas disponibilizadas pelo Spotify.

Projeto do Banco de Dados

=====

(a) Modelo Conceitual

O modelo conceitual desenvolvido foi construído a partir das informações reais presentes no dataset selecionado. Durante a modelagem foram identificadas quatro entidades principais: Musica, Artista, Album e Genero.

A entidade Musica detém os principais atributos relacionados à cada faixa musical, como identificador, nome, popularidade e tempo da música. As entidades Artista, Album e Genero foram definidas para representar as informações associadas a diferentes agrupamentos de músicas presentes no dataset.

Os relacionamentos foram definidos com base nas associações observadas nos dados. O relacionamento entre Artista e Musica possui cardinalidade muitos-para-muitos, n:n, uma vez que uma música pode possuir múltiplos artistas e um artista pode produzir várias músicas. Além disso, esse relacionamento possui o atributo *tipo_participacao*, que o Spotify utiliza para distinguir entre participações principais ou colaborações secundárias por artistas.

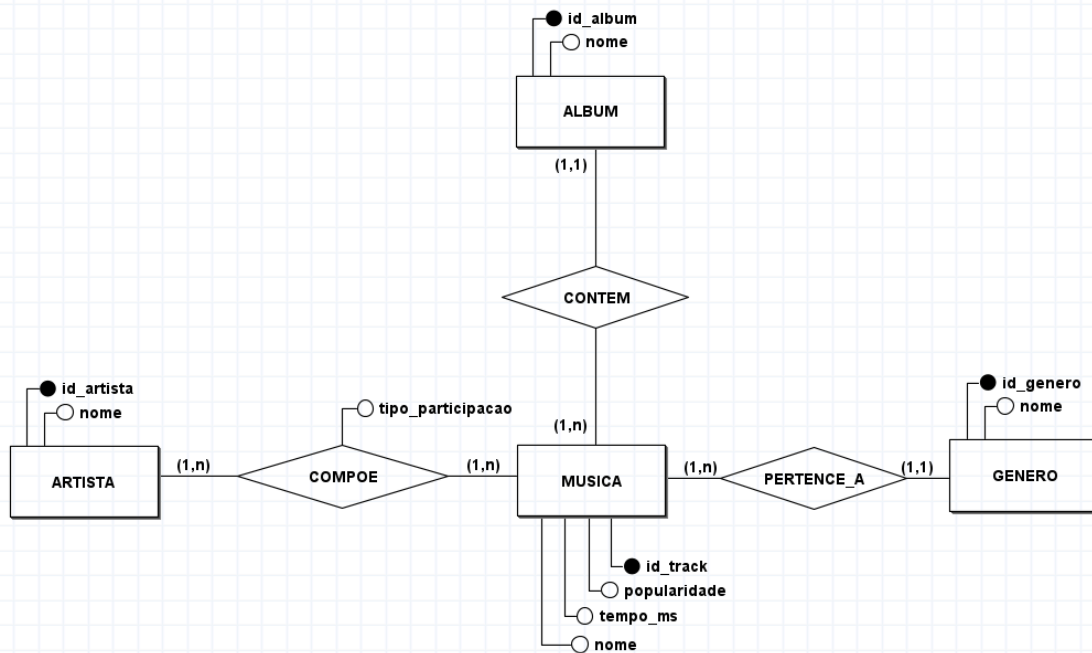
No modelo conceitual desenvolvido, foi definido um relacionamento entre Musica e Genero. Embora no contexto real uma música possa estar associada a múltiplos gêneros musicais, ao seguirmos o Spotify Tracks Dataset, cada faixa possui apenas um único gênero registrado no campo *track_genre*. Dessa forma, a cardinalidade adotada no modelo foi de 1:1 para Música em relação a Gênero. Na direção oposta da relação, um gênero pode estar associado a várias músicas, então teremos uma cardinalidade 1:n.

Também foram modelados os relacionamentos entre Album e Musica, e entre Genero e Musica, representando, respectivamente, o álbum ao qual cada

música pertence, e sua classificação em categorias arbitrárias pelo Spotify.

O diagrama entidade-relacionamento correspondente ao modelo conceitual, de acordo com o que foi escrito, segue abaixo:

Modelo E.R - Imagem



(b) Modelo Lógico

O modelo lógico foi desenvolvido a partir da conversão do modelo conceitual elaborado anteriormente, aplicando as regras de transformação entre entidades, relacionamentos e tabelas relacionais. Nessa etapa, as entidades identificadas no modelo conceitual foram convertidas em relações contendo seus respectivos atributos, chaves primárias e chaves estrangeiras.

As entidades Artista, Album, Genero e Musica foram convertidas diretamente em tabelas independentes, preservando os atributos definidos durante a etapa conceitual. A tabela Musica passou a armazenar também as chaves estrangeiras relacionadas às tabelas Album e Genero, mapeando as restrições e associações identificadas anteriormente.

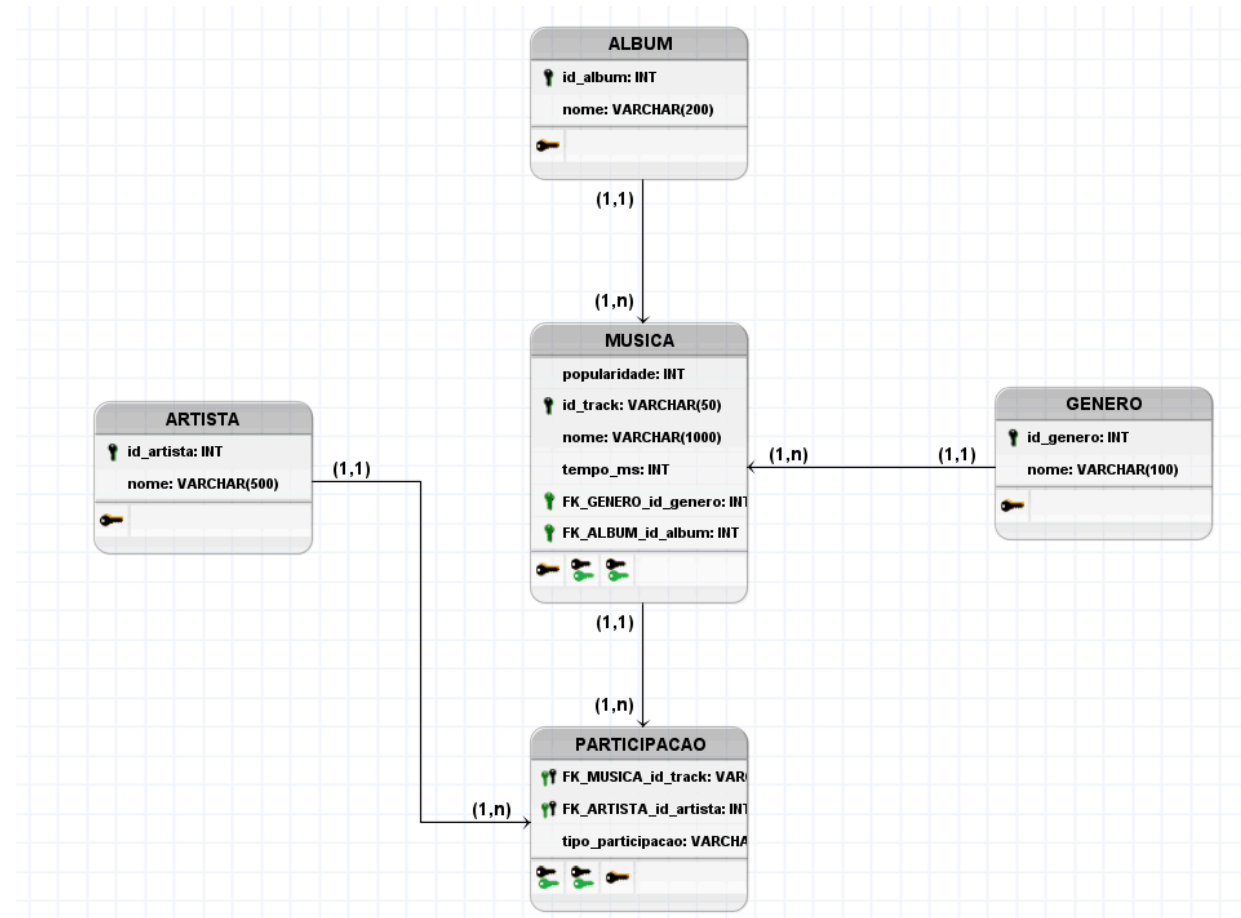
O relacionamento muitos-para-muitos entre Artista e Musica foi convertido para uma tabela intermediária denominada Participacao. Essa conversão foi necessária pois relacionamentos N:N não podem ser representados diretamente em modelos relacionais simples. Dessa forma, a tabela Participacao passou a armazenar as chaves estrangeiras das tabelas Artista e Musica, além do

atributo `tipo_participacao`, utilizado para distinguir diferentes formas de colaboração entre artistas.

As cardinalidades observadas no modelo conceitual foram preservadas durante a conversão para o modelo lógico, garantindo que as restrições existentes nos dados originais continuassem representadas corretamente na estrutura relacional construída.

O diagrama correspondente ao modelo lógico, contendo as relações, chaves primárias e chaves estrangeiras definidas, é apresentado a seguir:

Modelo Lógico - Imagem



O modelo físico foi desenvolvido a partir do modelo lógico previamente elaborado, utilizando o Sistema Gerenciador de Banco de Dados MySQL para implementação das estruturas relacionais e armazenamento dos dados.

Inicialmente, foi realizada uma etapa de preparação do conjunto de dados utilizado. O dataset original encontrava-se em formato CSV contendo informações relacionadas a músicas, artistas, álbuns e gêneros musicais. Antes da inserção no banco, foi necessário realizar um processo de limpeza e normalização dos dados visando reduzir inconsistências e adequar o conjunto às estruturas definidas no projeto do banco.

Durante a etapa de pré-processamento foram executadas operações de limpeza utilizando scripts em Python, incluindo:

- Remoção de colunas desnecessárias presentes no dataset original;
- Remoção de registros contendo valores nulos;
- Remoção de registros duplicados;
- Separação de atributos compostos, permitindo a normalização dos dados;
- Geração de arquivos CSV independentes para cada entidade do banco.

Após o processo de limpeza, o conjunto de dados apresentou redução no número de registros devido à remoção de duplicatas e inconsistências, permitindo maior integridade na importação.

Os dados foram então separados em arquivos distintos correspondentes às entidades definidas no modelo lógico:

GENERO | ARTISTA | ALBUM | MUSICA | PARTICIPACAO

Com os arquivos preparados, foi realizada a implementação física do banco utilizando comandos SQL para criação das tabelas, definição das chaves primárias e estabelecimento das chaves estrangeiras responsáveis por manter a integridade referencial entre as entidades.

A população do banco foi realizada através da importação dos arquivos CSV gerados durante a etapa de processamento. Para evitar violações de integridade referencial, as tabelas foram populadas obedecendo à ordem de dependência entre as entidades.

A ordem utilizada para inserção dos dados foi:

1. GENERO
2. ALBUM
3. ARTISTA
4. MUSICA
5. PARTICIPACAO

Após a conclusão da importação, foram executadas consultas de validação visando confirmar a correta inserção dos registros e o funcionamento adequado das estruturas criadas.

As consultas executadas incluem contagem total de registros e visualização dos primeiros elementos armazenados em cada tabela, permitindo verificar a consistência dos dados populados.

Tratamento do CSV

```
Linhas originais: 114000
Após remover nulos: 113999
Após remover duplicatas: 89740
```

```
      track_id      artists ... time_signature track_genre
0  5Su0ikwiRyPMVoIQDJUgSV      Gen Hoshino ...         4      acoustic
1  4qPNDBW1i3p13qLCt0Ki3A      Ben Woodward ...         4      acoustic
2  1iJBSr7s7jYXzM8EGcbK5b  Ingrid Michaelson;ZAYN ...         4      acoustic
3  6Lfxq3CG4xtTiEg7opyCyx      Kina Grannis ...         3      acoustic
4  5vjLSffimiIP26QG5WcN2K      Chord Overstreet ...         4      acoustic
```

```
[5 rows x 20 columns]
```

```
Quantidade generos:
113
```

```
   id_genero   nome
0          1  acoustic
1          2  afrobeat
2          3  alt-rock
3          4  alternative
4          5  ambient
5          6  anime
6          7  black-metal
7          8  bluegrass
8          9  blues
9         10  brazil
```

```
Quantidade albuns:
46589
```

```
   id_album   nome
0          1  Comedy
1          2  Ghost (Acoustic)
2          3  To Begin Again
3          4  Crazy Rich Asians (Original Motion Picture Sou...
4          5  Hold On
```

```
Quantidade artistas:
29858
```

```
   id_artista   nome
0          1  Gen Hoshino
1          2  Ben Woodward
2          3  Ingrid Michaelson
3          4  ZAYN
4          5  Kina Grannis
```

```
Quantidade musicas:
89740
```

```
   id_track   nome   popularidade   tempo_ms   FK_GENERO_id_genero   FK_ALBUM_id_album
0  5Su0ikwiRyPMVoIQDJUgSV      Comedy           73      230666           1           1
1  4qPNDBW1i3p13qLCt0Ki3A      Ghost - Acoustic       55      149610           1           2
2  1iJBSr7s7jYXzM8EGcbK5b      To Begin Again         57      210826           1           3
3  6Lfxq3CG4xtTiEg7opyCyx  Can't Help Falling In Love       71      201933           1           4
4  5vjLSffimiIP26QG5WcN2K      Hold On                82      198853           1           5
```

```
Quantidade participacoes:
123424
```

```
   FK_MUSICA_id_track   FK_ARTISTA_id_artista   tipo_participacao
0  5Su0ikwiRyPMVoIQDJUgSV           1      principal
1  4qPNDBW1i3p13qLCt0Ki3A           2      principal
2  1iJBSr7s7jYXzM8EGcbK5b           3      principal
3  1iJBSr7s7jYXzM8EGcbK5b           4      feat
4  6Lfxq3CG4xtTiEg7opyCyx           5      principal
```

```
CSVs exportados com sucesso.
```

Modelo Físico - BrModelo

```
/* spotify_bd_physical_model: */

CREATE TABLE GENERO (
    id_genero INT PRIMARY KEY,
    nome VARCHAR(100)
);

CREATE TABLE ARTISTA (
    id_artista INT PRIMARY KEY,
    nome VARCHAR(500)
);

CREATE TABLE MUSICA (
    popularidade INT,
    id_track VARCHAR(50) PRIMARY KEY,
    nome VARCHAR(1000),
    tempo_ms INT,
    FK_GENERO_id_genero INT,
    FK_ALBUM_id_album INT
);

CREATE TABLE ALBUM (
    id_album INT PRIMARY KEY,
    nome VARCHAR(200)
);

CREATE TABLE PARTICIPACAO (
    FK_MUSICA_id_track VARCHAR(50),
    FK_ARTISTA_id_artista INT,
    tipo_participacao VARCHAR(50),
    PRIMARY KEY (FK_MUSICA_id_track, FK_ARTISTA_id_artista)
);

ALTER TABLE MUSICA ADD CONSTRAINT FK_MUSICA_2
    FOREIGN KEY (FK_GENERO_id_genero)
    REFERENCES GENERO (id_genero)
    ON DELETE RESTRICT;

ALTER TABLE MUSICA ADD CONSTRAINT FK_MUSICA_3
    FOREIGN KEY (FK_ALBUM_id_album)
    REFERENCES ALBUM (id_album)
    ON DELETE RESTRICT;

ALTER TABLE PARTICIPACAO ADD CONSTRAINT FK_PARTICIPACAO_1
    FOREIGN KEY (FK_MUSICA_id_track)
    REFERENCES MUSICA (id_track)
    ON DELETE RESTRICT;

ALTER TABLE PARTICIPACAO ADD CONSTRAINT FK_PARTICIPACAO_2
    FOREIGN KEY (FK_ARTISTA_id_artista)
    REFERENCES ARTISTA (id_artista)
    ON DELETE RESTRICT;
```

Estrutura Física das Tabelas

GENERO

- id_genero (PK)
- nome

ARTISTA

- id_artista (PK)
- nome

ALBUM

- id_album (PK)
- nome

MUSICA

- id_track (PK)
- nome
- popularidade
- tempo_ms
- FK_GENERO_id_genero (FK)
- FK_ALBUM_id_album (FK)

PARTICIPACAO

- FK_MUSICA_id_track (PK + FK)
- FK_ARTISTA_id_artista (PK + FK)
- tipo_participacao

Evidências do Banco de dados

Criação, acesso e tabelas

```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| catalogo_filmes |
| information_schema |
| mysql |
| performance_schema |
| spotify_db |
| sys |
+-----+
6 rows in set (0.00 sec)

mysql> USE spotify_db;
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_spotify_db |
+-----+
| album |
| artista |
| genero |
| musica |
| participacao |
+-----+
5 rows in set (0.00 sec)
```

Detalhes das tabelas

```
mysql> DESCRIBE genero;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id_genero | int | NO | PRI | NULL | |
| nome | varchar(100) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
```

```
mysql> DESCRIBE artista;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id_artista | int | NO | PRI | NULL | |
| nome | varchar(500) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> DESCRIBE album;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id_album | int | NO | PRI | NULL | |
| nome | varchar(500) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> DESCRIBE participacao;
```

Field	Type	Null	Key	Default	Extra
FK_MUSICA_id_track	varchar(50)	NO	PRI	NULL	
FK_ARTISTA_id_artista	int	NO	PRI	NULL	
tipo_participacao	varchar(50)	YES		NULL	

3 rows in set (0.00 sec)

```
mysql> DESCRIBE musica;
```

Field	Type	Null	Key	Default	Extra
popularidade	int	YES		NULL	
id_track	varchar(50)	NO	PRI	NULL	
nome	varchar(1000)	YES		NULL	
tempo_ms	int	YES		NULL	
FK_GENERO_id_genero	int	YES	MUL	NULL	
FK_ALBUM_id_album	int	YES	MUL	NULL	

6 rows in set (0.00 sec)

Evidência de população

```
mysql> SELECT COUNT(*) FROM genero;
```

COUNT(*)
113

1 row in set (0.00 sec)

```
mysql> SELECT * FROM genero LIMIT 10;
```

id_genero	nome
1	acoustic
2	afrobeat
3	alt-rock
4	alternative
5	ambient
6	anime
7	black-metal
8	bluegrass
9	blues
10	brazil

10 rows in set (0.00 sec)

```
mysql> SELECT COUNT(*) FROM album;
```

COUNT(*)
46589

1 row in set (0.00 sec)

```
mysql> SELECT * FROM album LIMIT 10;
```

id_album	nome
1	Comedy
2	Ghost (Acoustic)
3	To Begin Again
4	Crazy Rich Asians (Original Motion Picture Soundtrack)
5	Hold On
6	Days I Will Remember
7	Is There Anybody Out There?
8	We Sing. We Dance. We Steal Things.
9	Hunger
10	Episode

10 rows in set (0.00 sec)

```
mysql> SELECT COUNT(*) FROM artista;
+-----+
| COUNT(*) |
+-----+
|      29858 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM artista LIMIT 10;
+-----+-----+
| id_artista | nome |
+-----+-----+
|          1 | Gen Hoshino |
|          2 | Ben Woodward |
|          3 | Ingrid Michaelson |
|          4 | ZAYN |
|          5 | Kina Grannis |
|          6 | Chord Overstreet |
|          7 | Tyrone Wells |
|          8 | A Great Big World |
|          9 | Christina Aguilera |
|         10 | Jason Mraz |
+-----+-----+
10 rows in set (0.00 sec)
```

```
mysql> SELECT COUNT(*) FROM musica;
+-----+
| COUNT(*) |
+-----+
|      89740 |
+-----+
1 row in set (0.01 sec)

mysql> SELECT * FROM musica LIMIT 10;
+-----+-----+-----+-----+-----+-----+
| popularidade | id_track | nome | tempo_ms | FK_GENERO_id_genero | FK_ALBUM_id_album |
+-----+-----+-----+-----+-----+-----+
|          44 | 0000vdREvCVMxbQTkS888c | Lolly | 160725 | 40 | 19251 |
|          47 | 000CC8EParg640mTxVnZ0p | It's All Coming Back To Me Now (Glee Cast Version) | 322933 | 18 | 8887 |
|          22 | 000Iz0K615UepwSJ5z2RE5 | Bóxig Leise - Pig & Dan Remix | 515360 | 74 | 20096 |
|          19 | 000qpd0c97IMTBvF8gwcpy | Tíef | 331240 | 74 | 26140 |
|          62 | 000RDCYioLteXcut0jewey | Teeje Week | 190203 | 52 | 24335 |
|          24 | 0017XiMkqbTfF2AU0zlhj6 | Thanksgiving Chicken | 127040 | 19 | 9486 |
|           0 | 001APMD0L3qtx1526T11n1 | Better | 176320 | 16 | 1826 |
|          30 | 001pyq8FLNSL1C8orNLI0b | Poor Man | 214600 | 8 | 3817 |
|          38 | 001YQlnDSduXdSLgBd66gT | EL Tiempo Es Dinero - Remasterizado 2007 | 177266 | 86 | 36802 |
|          17 | 002qpSULhHAw60GqFxba01 | Love Generation | 410666 | 47 | 22288 |
+-----+-----+-----+-----+-----+-----+
10 rows in set (0.00 sec)
```

```
mysql> SELECT COUNT(*) FROM participacao;
+-----+
| COUNT(*) |
+-----+
|      123424 |
+-----+
1 row in set (0.02 sec)

mysql> SELECT * FROM participacao LIMIT 10;
+-----+-----+-----+
| FK_MUSICA_id_track | FK_ARTISTA_id_artista | tipo_participacao |
+-----+-----+-----+
| 0000vdREvCVMxbQTkS888c | 14057 | principal |
| 000CC8EParg640mTxVnZ0p | 5830 | principal |
| 000Iz0K615UepwSJ5z2RE5 | 8518 | feat |
| 000Iz0K615UepwSJ5z2RE5 | 11602 | principal |
| 000qpd0c97IMTBvF8gwcpy | 11602 | principal |
| 000RDCYioLteXcut0jewey | 16920 | principal |
| 0017XiMkqbTfF2AU0zlhj6 | 6246 | principal |
| 001APMD0L3qtx1526T11n1 | 4748 | principal |
| 001APMD0L3qtx1526T11n1 | 4769 | feat |
| 001pyq8FLNSL1C8orNLI0b | 2260 | principal |
+-----+-----+-----+
10 rows in set (0.00 sec)
```

Consultas

Consulta nº 1

```
USE spotify_db;
-- =====
--          Consulta nº 1
-- =====
-- Consulta Principal, Retorna todas as informações.
-- *****
SELECT MS.nome AS Musica,
       ART.nome AS Artista,
       ALB.nome AS Album,
       GEN.nome AS Genero,
       CONCAT(FLOOR(MS.tempo_ms / 60000), 'm',
              LPAD(FLOOR((MS.tempo_ms % 60000) / 1000), 2, '0'), 's') AS
duracao,
       MS.popularidade AS popularidade,
       PT.tipo_participacao AS Atuacao
FROM musica as MS
JOIN album AS ALB ON MS.FK_ALBUM_id_album = ALB.id_album
JOIN participacao as PT ON MS.id_track = PT.FK_MUSICA_id_track
JOIN artista as ART ON ART.id_artista = PT.FK_ARTISTA_id_artista
JOIN genero as GEN ON GEN.id_genero = MS.FK_GENERO_id_genero
WHERE MS.nome LIKE "%%"
ORDER BY MS.popularidade DESC
;
```

Musica	Artista	Album	Genero	duracao	popularidade	Atu
Unholy (feat. Kim Petras)	Sam Smith	Unholy (feat. Kim Petras)	dance	2m36s	100	princ
Unholy (feat. Kim Petras)	Kim Petras	Unholy (feat. Kim Petras)	dance	2m36s	100	feat
Quevedo: Bzrp Music Sessions, Vol. 52	Bizarrap	Quevedo: Bzrp Music Sessions, Vol. 52	hip-hop	3m18s	99	princ
Quevedo: Bzrp Music Sessions, Vol. 52	Quevedo	Quevedo: Bzrp Music Sessions, Vol. 52	hip-hop	3m18s	99	feat

Esta consulta é responsável por listar as músicas cadastradas juntamente com suas principais informações, incluindo artista, álbum, gênero, duração, popularidade e tipo de participação do artista na faixa. Ela representa a consulta principal da aplicação, servindo como base para os filtros e ordenações disponíveis na interface.

Consulta n° 2

```
USE spotify_db;
-- =====
--          Consulta n° 2
-- =====
-- Consulta Secundária, Retorna as estatísticas dos artistas
-- *****
SELECT ART.nome AS artista,
       AVG(MS.popularidade) AS popularidade,
       COUNT(MS.id_track) AS qtd_musicas,
       COUNT(DISTINCT ALB.id_album) AS qtd_album
FROM musica as MS
JOIN album as ALB ON MS.FK_ALBUM_id_album = ALB.id_album
JOIN participacao AS PT ON MS.id_track = PT.FK_MUSICA_id_track
JOIN artista AS ART ON PT.FK_ARTISTA_id_artista = ART.id_artista
GROUP BY ART.id_artista
HAVING ART.nome LIKE "%%"
LIMIT 5 OFFSET 0
;
```

	artista	popularidade	qtd_musicas	qtd_album
▶	Gen Hoshino	51.5714	7	6
	Ben Woodward	41.2222	9	9
	Ingrid Michaelson	49.0833	12	10
	ZAYN	67.1111	9	8
	Kina Grannis	52.5238	21	21

Esta consulta retorna estatísticas gerais de cada artista, apresentando sua popularidade média, quantidade de músicas e quantidade de álbuns em que participa. Essas informações serão utilizadas na página de detalhes do artista.

Consulta n° 3

```
USE spotify_db;
-- =====
--          Consulta n° 3
-- =====
-- Consulta Secundária, Retorna as 5 músicas mais populares do
-- artista
-- *****
SELECT ART.nome AS artista,
       MS.nome AS musica,
       MS.popularidade AS popularidade
```

```

FROM musica AS MS
JOIN participacao AS PT ON MS.id_track = PT.FK_MUSICA_id_track
JOIN artista AS ART ON PT.FK_ARTISTA_id_artista = ART.id_artista
WHERE ART.nome LIKE "%Rihanna%"
ORDER BY MS.popularidade DESC
LIMIT 5
;

```

artista	musica	popularidade
Rihanna	Love The Way You Lie	83
Rihanna	This Is What You Came For	81
Rihanna	Can't Remember to Forget You (feat. Rihanna)	75
Rihanna	Break It Off (feat. Rihanna)	56
Rihanna	Shy Ronnie 2: Ronnie & Clyde	49

A consulta recupera as músicas mais populares de um determinado artista, ordenando-as por popularidade de forma decrescente. Na aplicação, será utilizada para apresentar rapidamente os principais sucessos do artista selecionado.

Consulta nº 4

```

USE spotify_db;
-- =====
--                Consulta nº 4
-- =====
-- Consulta Secundária, Retorna estatísticas dos gêneros
-- *****
SELECT GEN.nome AS genero,
       COUNT(DISTINCT MS.id_track) AS qtd_musicas,
       AVG(MS.popularidade) AS pop_media,
       COUNT(DISTINCT ART.id_artista) AS qtd_artistas
FROM musica AS MS
JOIN genero AS GEN ON MS.FK_GENERO_id_genero = GEN.id_genero
JOIN participacao AS PT ON MS.id_track = PT.FK_MUSICA_id_track
JOIN artista AS ART ON PT.FK_ARTISTA_id_artista = ART.id_artista
GROUP BY GEN.id_genero
HAVING GEN.nome LIKE "%%"
ORDER BY GEN.nome ASC;

```

genero	qtd_musicas	pop_media	qtd_artistas
acoustic	1000	42.7109	348
afrobeat	999	24.6361	387
alt-rock	999	33.1277	351
alternative	407	22.3424	130
ambient	999	41.8547	393

Esta consulta apresenta estatísticas referentes aos gêneros musicais cadastrados, incluindo quantidade de músicas, popularidade média e quantidade de artistas associados. Seus resultados alimentarão a aba de gêneros da aplicação.

Consulta n° 5

```
USE spotify_db;
-- =====
--           Consulta n° 5
-- =====
-- Consulta Secundária, Retorna as estatísticas de um álbum
-- *****
SELECT ALB.nome AS album,
       COUNT(DISTINCT MS.id_track) AS qtd_musicas,
       AVG(MS.popularidade) AS popularidade_media,
       COUNT(DISTINCT ART.id_artista) AS qtd_artistas
FROM album AS ALB
LEFT JOIN musica AS MS ON ALB.id_album = MS.FK_ALBUM_id_album
LEFT JOIN participacao AS PT ON MS.id_track =
PT.FK_MUSICA_id_track
LEFT JOIN artista AS ART ON PT.FK_ARTISTA_id_artista =
ART.id_artista
GROUP BY ALB.id_album
HAVING ALB.nome LIKE "%%"
ORDER BY qtd_musicas DESC
LIMIT 5
;
```

album	qtd_musicas	popularidade_media	qtd_artistas
The Complete Hank Williams	110	16.2774	6
Greatest Hits	77	40.8660	45
Mozart: A Night of Classics	75	15.2426	57
Alternative Christmas 2022	73	0.0000	55
Mozart - All Day Classics	68	12.3351	68

A consulta apresenta informações estatísticas sobre cada álbum, incluindo quantidade de músicas, popularidade média e número de artistas participantes. Na aplicação, servirá para exibir um resumo completo dos álbuns cadastrados.

Consulta n° 6

```
USE spotify_db;
-- =====
--                Consulta n° 6
-- =====
-- Consulta Secundária, retorna Artistas com a média maior de
músicas
-- *****
SELECT ART.nome, COUNT(*) AS total_musicas
FROM artista ART
JOIN participacao PT
    ON ART.id_artista = PT.FK_ARTISTA_id_artista
GROUP BY ART.id_artista, ART.nome
HAVING COUNT(*) >
(
    SELECT AVG(qtd)
    FROM (
        SELECT COUNT(*) AS qtd
        FROM participacao
        GROUP BY FK_ARTISTA_id_artista
    ) AS media_artistas
)
ORDER BY total_musicas DESC
LIMIT 5
;
```

nome	total_musicas
George Jones	332
Pritam	323
Wolfgang Amadeus Mozart	305
Arijit Singh	259
Hank Williams	243

Esta consulta identifica os artistas cuja quantidade de músicas é superior à média de todos os artistas presentes na base de dados. A utilização da subconsulta permite calcular dinamicamente essa média, destacando os artistas com maior produção musical.

Consulta n° 7

```
USE spotify_db;
-- =====
--          Consulta n° 7
-- =====
-- Consulta Secundária, retorna a música mais popular de cada
gênero
-- *****
SELECT
    GEN.nome AS genero,
    MS.nome AS musica,
    MS.popularidade
FROM musica MS
JOIN genero GEN
ON GEN.id_genero = MS.FK_GENERO_id_genero
JOIN (
    SELECT
        FK_GENERO_id_genero,
        MAX(popularidade) AS max_pop
    FROM musica
    GROUP BY FK_GENERO_id_genero
) ranking
ON ranking.FK_GENERO_id_genero = MS.FK_GENERO_id_genero
AND ranking.max_pop = MS.popularidade
ORDER BY GEN.nome
LIMIT 5
;
```

genero	musica	popularidade
acoustic	Hold On	82
afrobeat	Atrévete-Te-Te	75
alt-rock	Sweater Weather	93
alternative	Miss You	87
ambient	Apocalypse	84

Esta consulta retorna a música de maior popularidade pertencente a cada gênero musical. A subconsulta é utilizada para determinar a maior popularidade de cada gênero, permitindo identificar automaticamente seus principais destaques.

Aplicação

Objetivo

A aplicação desenvolvida, batizada de Spotify Explorer, é uma plataforma Web interativa cujo objetivo principal é permitir a exploração, visualização e análise estatística de um ecossistema de dados musicais extraídos do *Spotify Tracks Dataset*. Focada estritamente na leitura de dados (conforme os requisitos de escopo do projeto), a aplicação fornece ao usuário ferramentas dinâmicas de busca, filtragem e agregação de dados em tempo real.

Arquitetura do Sistema

O sistema foi projetado seguindo o padrão de arquitetura em 3 camadas (*3-tier architecture*), garantindo a modularidade, escalabilidade e total independência entre a interface do usuário, a lógica de negócios e o armazenamento dos dados.

Interface do Usuário	HTTP/JSON	Servidor Backend	SQLAlchemy/TCP	Banco de Dados
Node.js + Vite + HTML/CSS	<--- --->	Python + Flask	<--- --->	MySQL na Nuvem/Local

1.Camada de Apresentação (Frontend): Construída utilizando Node.js com o *bundler* Vite, além de tecnologias nativas como HTML5 e CSS3. Essa camada gerencia a interface gráfica e a experiência do usuário, capturando parâmetros de busca e interações.

2.Camada de Lógica de Negócios (Backend): Desenvolvida em Python utilizando o microframework Flask. O backend atua como uma API RESTful, responsável por receber as requisições da interface, aplicar regras de validação, intermediar a comunicação com o banco de dados e devolver as respostas estruturadas em formato JSON.

3.Camada de Dados (Persistência): Composta por um Sistema Gerenciador de Banco de Dados MySQL. A comunicação entre o código Python e o banco de dados é intermediada pelo ORM SQLAlchemy em

conjunto com o driver nativo PyMySQL, permitindo a execução eficiente e segura das consultas relacionais.

Fluxo de Execução de Requisições

O ciclo de vida de uma consulta na aplicação segue um fluxo síncrono e integrado:

- O usuário interage com a interface (por exemplo, pesquisando por um artista ou filtrando por um gênero musical).
- O Frontend (Vite/Node) dispara uma requisição HTTP assíncrona contendo os parâmetros de busca para o servidor backend.
- O Servidor Backend (Flask) processa os parâmetros e aciona a camada de persistência.
- As consultas SQL otimizadas (formuladas na Parte 3 deste projeto) são submetidas e executadas diretamente no servidor MySQL.
- O banco de dados retorna o conjunto de registros para o Flask, que os estrutura e envia de volta ao Frontend através de uma resposta JSON.
- O Frontend recebe os dados brutos e atualiza dinamicamente os componentes visuais e estatísticos na tela do usuário, dispensando a necessidade de recarregar a página.

Infraestrutura e Deploy na Nuvem

Para simular um ambiente de produção real e descentralizado do ambiente local, todas as camadas da aplicação foram implantadas na nuvem:

- Banco de Dados: Hospedado de forma gerenciada na nuvem através da plataforma Aiven, garantindo alta disponibilidade e conexão remota segura via TCP/IP.
- Backend e Frontend: Hospedados e executados continuamente na plataforma Render, que realiza o build automatizado da aplicação visual e do servidor Python.

Todo o ciclo de vida do desenvolvimento foi controlado utilizando o sistema de versionamento Git e centralizado em um repositório na plataforma GitHub, onde se encontram toda a árvore de diretórios do projeto, o histórico de *commits* e a documentação técnica complementar.

Tecnologias Utilizadas

A tabela abaixo consolida o conjunto de ferramentas e tecnologias empregadas no ecossistema do projeto:

Camada / Contexto	Tecnologia / Ferramenta
Modelagem e Design	brModelo, MySQL Workbench, DBeaver
Banco de Dados (SGBD)	MySQL (Instância Cloud via Aiven)
Ambiente Backend	Python 3, Flask, SQLAlchemy, PyMySQL
Ambiente Frontend	Node.js, Vite, HTML5, CSS3
Controle de Versão e Infra	Git, GitHub, Render

Considerações Finais

=====

O desenvolvimento do projeto Spotify Explorer cumpriu com êxito todas as metas pedagógicas estabelecidas pela disciplina de Banco de Dados I. Mais do que consolidar a teoria vista em sala de aula, a oportunidade de implementar um sistema completo e levá-lo a um cenário produtivo real provou-se uma etapa enriquecedora e altamente relevante para a formação acadêmica e técnica em Ciência da Computação.

O grande diferencial deste trabalho residiu na fidelidade ao ciclo de vida real da Engenharia de Software. Percorrer sequencialmente a análise exploratória de dados brutos (Kaggle), a modelagem conceitual abstrata, o mapeamento lógico, a normalização física, a implementação de APIs e a implantação distribuída na nuvem consolidou um fluxo prático abrangente. A execução do projeto envolveu o enfrentamento de desafios típicos do mercado, como latência de rede em conexões remotas de banco de dados, tratamento de concorrência e a necessidade de construção de queries SQL otimizadas e indexadas para evitar a sobrecarga dos recursos da aplicação.

Conclui-se que o saldo do projeto é amplamente positivo. A aplicação não apenas validou o modelo relacional projetado na Parte 2, como também materializou a capacidade de transformar dados massivos e

desestruturados em informações visuais legíveis e de alto valor prático. Os conceitos sedimentados ao longo desta trajetória técnica servem como pilares fundamentais para o exercício profissional e para o desenvolvimento de sistemas de software complexos.

Referências e Links

Abaixo estão listadas as referências técnicas, ferramentas de software e bases de dados que serviram de alicerce para a pesquisa, modelagem e codificação de todo o projeto:

1. KAGGLE. *Spotify Tracks Dataset*. Disponível em: <https://www.kaggle.com/datasets/maharshifichadia/spotify-tracks-dataset>. (Base de dados bruta em formato CSV utilizada para a população do SGBD).
2. PYTHON SOFTWARE FOUNDATION. *Flask Documentation (v3.0.x)*. Disponível em: <https://flask.palletsprojects.com/>. (Documentação do microframework utilizado no desenvolvimento da API REST).
3. SQLALCHEMY ORM. *SQLAlchemy 2.0 Documentation*. Disponível em: <https://www.sqlalchemy.org/>. (Referência técnica para o mapeamento objeto-relacional e abstração de consultas).
4. VITE SPA BUNDLER. *Vite.js - Guide & Documentation*. Disponível em: <https://vitejs.dev/>. (Documentação do ecossistema de build rápido utilizado no Frontend).
5. MYSQL SERVER. *MySQL 8.0 Reference Manual*. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. (Manual técnico para otimização de queries, joins e integridade referencial).

Links do Projeto

- Repositório do Código Fonte (GitHub):
<https://github.com/Rebornned/spotify-explorer>
- Aplicação Publicada (Render):
<https://spotify-frontend-09cs.onrender.com/>